RIT | CSCI 635 | April 22, 2026

Predicting Burnout in Remote Work Settings

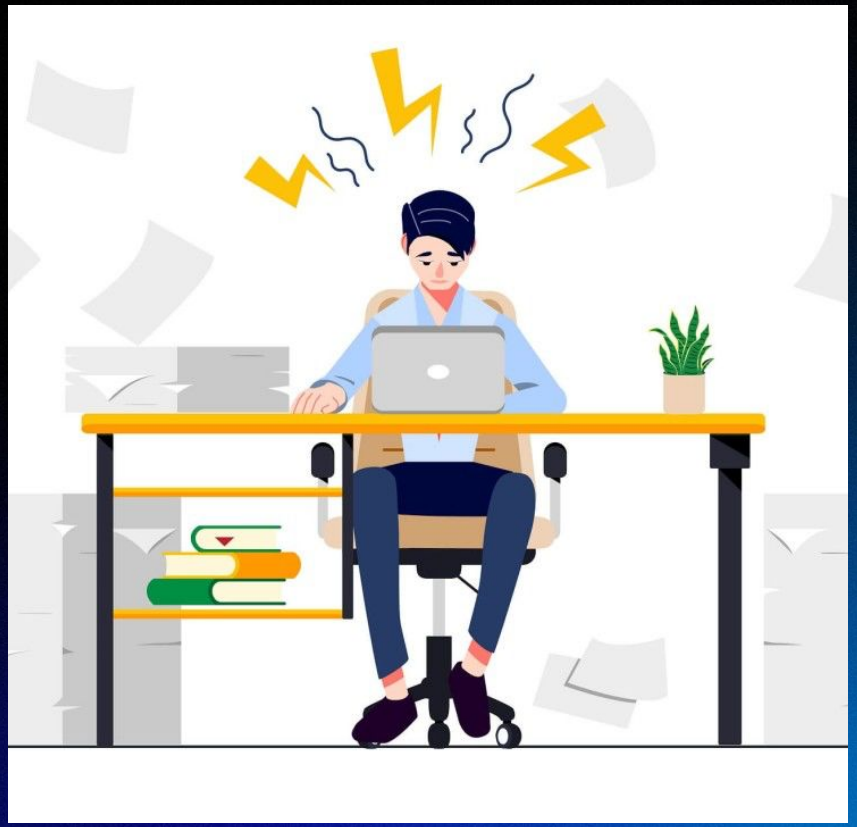
Vrutant Chaudhari
–
Will Hoover
–
Rina Peshori

Problem

Since the COVID pandemic, remote work rates in most industries have doubled or tripled (BLS).

Remote work poses a risk for social isolation and burnout previously unencountered in in-person settings.

It would be ideal to identify a risk of burnout before anything happens and prevent it by addressing the highest risk factors.



Project Goals

- Build machine learning models to predict burnout risk given several factors
- Validate, test, and fine-tune models to select best single or ensemble model
- Identify highest risk factors (if applicable)
- **Future:** use our identified model on real collected data to help people identify their personal risk

Dataset

Remote Work Burnout & Social Isolation (2026)

https://www.kaggle.com/datasets/aryanmdev/remote-work-burnout-and-social-isolation-2026?select=wfh_burnout_dataset.csv

Overview

- 2,000 records (+ our augmentation)
- 45-40-15 percent split
- **80-86% expected accuracy** for random forest/XGBoost (given)

Features

- ID: user_id
- Auto-logged: work/screen time hours, meeting count, app switches
- Self-reported: sleep hours, isolation index, fatigue score, breaks taken
- Target: burnout risk (low, medium, high)

Preliminary Analysis

- Mostly numerical features
- Significant class imbalance with High burnout risk minority
- Not enough given records for neural network

Preprocessing Tasks

Feature selection

- Deleted unique user ID
- Deleted regression target burnout_score

Scaling & Encoding

- Binary encoding on day_type (weekend or weekday)
- Z-score scaling on all other features

Augmentation

- Synthetic generation of unique sample data
- Engineered to closely resemble given data with appropriate class labelling

Other considerations

- Found no likely duplicates in preliminary analysis
- No data was missing

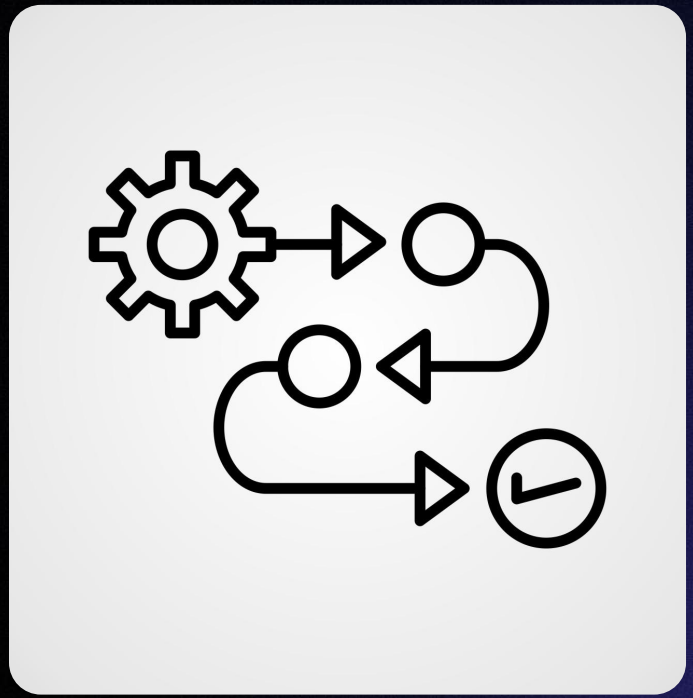
Data Augmentation

- Goal: Double the size of the original dataset
  - 2000 organic + 2000 synthetic = 4000 total samples
- Original class distribution is roughly maintained
  - Minority class remains a significant minority
  - We wanted to maintain the same challenges posed by the original dataset
- Data generation process:
  - Uses Grid Search with a Kernel Density estimation method to find best parameters using cross-validation
  - Does this separately for each class
- Can be based upon either unscaled, original data or pre-scaled data
  - If using unscaled, constraints are applied to enforce given categorical/int/float ranges for each feature

```
Original class distribution:
burnout_risk
0    0.509375
1    0.421250
2    0.069375
Name: proportion, dtype: float64

Scaled synthetic class distribution:
0    0.5095
1    0.4215
2    0.0690
Name: proportion, dtype: float64

Unscaled synthetic class distribution:
0    0.5095
1    0.4215
2    0.0690
Name: proportion, dtype: float64
```



The Process

- One or all models run through main file
- In-code models follow basic interface
  - Provide train function that takes training x, y sets
  - Train function returns classifier
- Can be run with or without synthetically generated data
- Pipeline also provides option to run models using custom implementation of k-fold cross-validation
- Automatically generates metric reports

Models

Goal: train high-performing models while maximizing from-scratch implementation

01
Logistic Regression

02
Random Forest

03
MLP

Logistic Regression

- Training done using BGD (full batch size used each epoch)
- Predicts probability of each sample belonging to a certain class
  - Uses One vs. Rest strategy to predict probability for each class
    - Trains separate classifier for each risk class ∈ {“Low”, “Medium”, “High”}
    - Needed for multi-class classification problem
  - Takes maximum of binary classifier results as final class prediction
  - Utilizes cross-entropy loss to measure difference between predicted probabilities and actual class labels & penalize incorrect predictions
- Uses class weighting to assign more weight to minority class during training
- Some hyperparameters:
  - max_iterations = 5000, $\eta = 0.01$, threshold (for convergence) = 1e-4

Random Forest

- N trees constructed using bootstrap sampling
- Splitting features selected by minimizing cross-entropy
- Early stopping - branching stops when:
  - Max depth reached (max_depth)
  - Number of samples on the branch goes under threshold (min_samples)
  - Majority class percentage exceeds threshold (decision_thresh)
- Hyperparameter values tuned using grid search + 10-fold cross validation
  - N = 50
  - Max_depth = 8, min_samples = 20, decision_thresh = 0.92

Multi-Layer Perceptron (MLP)

- Implemented a feedforward Multi-layer Perceptron (MLP) using PyTorch for multi-class classification
- Architecture consist of:
  - Input layer
  - One hidden layer with 32 neurons and ReLU activation
  - Output layer with 3 neurons corresponding to classes {Low, Medium, High}
- Model trained using mini-batch gradient descent (batch size = 64)
- Optimized using Adam optimizer with learning rate $\eta = 0.001$
- Uses CrossEntropyLoss to penalize incorrect predictions

Multi-Layer Perceptron (MLP) Continued…

- Incorporates multiple regularization techniques:
  - Dropout (rate = 0.3)
  - L1 regularization (λ = 1e-5)
  - L2 regularization (λ = 1e-4)
- Supports Early Stopping with patience = 15 to prevent overfitting
- Model trained for up to 150 epochs with validation monitoring
- Multiple MLP variants are trained, and the best model is selected based on validation performance:
  - Highest validation accuracy
  - Lowest validation loss (tie-breaker)

Results

***Without* Data Augmentation…**

Simple 80-20 Train-Test Split

	Training Accuracy	Test Accuracy	Precision	Recall	F1
Logistic Regression	0.8700	0.8825	0.82	0.88	0.85
Random Forest	0.9544	0.9600	0.96	0.96	0.96
MLP	0.9850	0.9850	0.99	0.98	0.99

Weighted averages of per-class metrics used where applicable.

K-Fold Cross Validation (k=10)

	Avg. Training Accuracy	Avg. Validation Accuracy
Logistic Regression	0.9192	0.9113
Random Forest	0.9576	0.9256
MLP	0.9849	0.9644

Results

***With* Data Augmentation…**

Simple 80-20 Train-Test Split

	Training Accuracy	Test Accuracy	Precision	Recall	F1
Logistic Regression	0.7566	0.8175	0.83	0.82	0.82
Random Forest	0.9300	0.9525	0.95	0.95	0.95
MLP	0.9316	0.9525	0.95	0.95	0.95

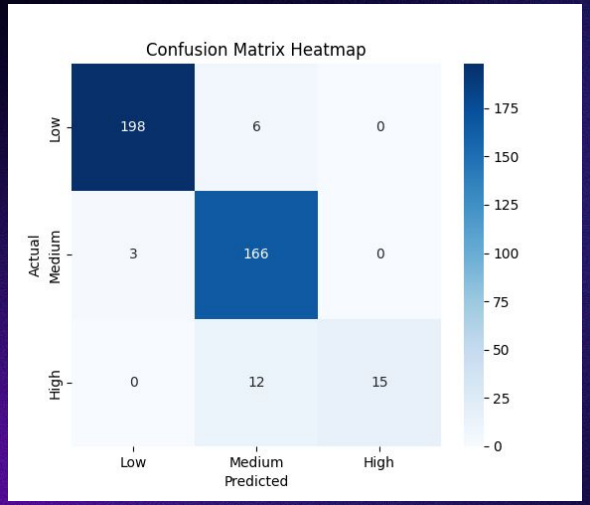
Weighted averages of per-class metrics used where applicable.

K-Fold Cross Validation (k=10)

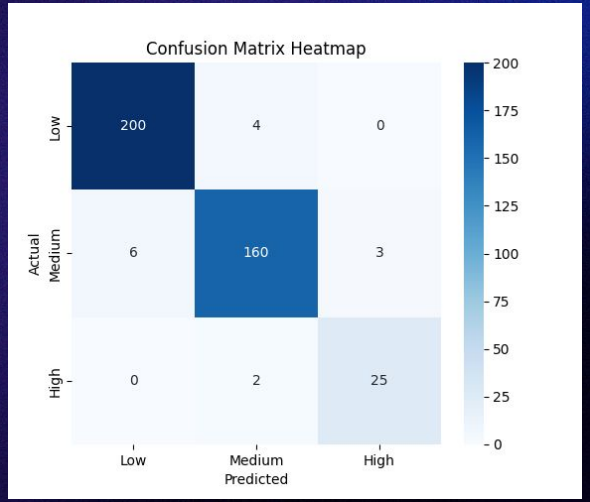
	Avg. Training Accuracy	Avg. Validation Accuracy
Logistic Regression	0.8090	0.8325
Random Forest	0.9347	0.9356
MLP	0.9385	0.9438

Results

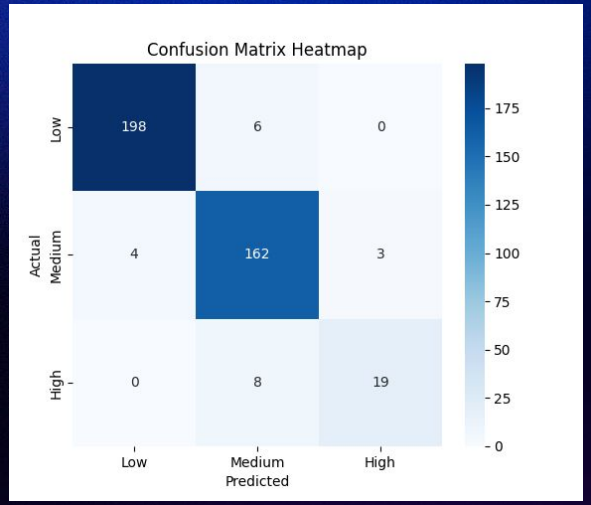
***With* Data Augmentation…**



Logistic Regression



Random Forest



MLP

Confusion matrices across all models were very similar with or without data augmentation, with the exception of Logistic Regression, which reported lower misclassification of the minority class when synthetically generated data was included.

Conclusions

Best Model?

In terms of performance, best model was MLP, with all metrics consistently above 90%, even when looking at per-class metrics or macro averages. These numbers remained relatively stable with data augmentation.

...Vs Expected

As compared to the 80-86% expected accuracy for Random Forest models run on this dataset, our best model significantly outperforms this metric, even when additional synthetic data is included. Our own Random Forest also significantly outperforms this metric, and Logistic Regression meets the expectations of its more complex counterparts.

Other Insights

Most important attributes from Random Forest were fatigue_score and sleep_hours.

Opportunities for Future Work

- Limitation addressal
  - More than half of data relies on self-reported metrics, which can be subjective
  - Some samples provided are from the same user across different days
    - Could lead to overly similar data between training/validation/test
  - Data had to be augmented - more organic collection would provide better results
  - Could further address minority class issues by implementing SMOTE or similar

- Our model(s) can be applied on real-world data to help prevent burnout!
  - By knowing your risk, you can ensure you are taking the right steps to prevent further decline.

You can fill out this survey, accessible via the QR code, to receive your own prediction results from each of our three models! →

Thank you!

– ~ – Vrutant Chaudhari, Will Hoover, Rina Peshori – ~ –